

Documentation Technique — Vite & Gourmand

1. *Réflexions initiales technologiques*

Contexte du projet

Vite & Gourmand est une entreprise de traiteur fondée en 2001 à Bordeaux par Julie Martin et José Fernandez. L'objectif est de développer une application web permettant d'afficher les menus et de gérer les commandes en ligne, remplaçant ainsi l'envoi manuel par email.

Choix technologiques

Figma — Design et prototypage

Figma a été utilisé pour concevoir les maquettes desktop et mobile ainsi que générer une ossature React initiale. Le code exporté a ensuite été entièrement adapté, refactorisé et enrichi avec les fonctionnalités métier (authentification, commandes, BDD, sécurité).

Frontend — React + TypeScript + Vite

React a été choisi pour sa capacité à créer des interfaces utilisateur dynamiques et réactives. TypeScript apporte la sécurité du typage statique, réduisant les erreurs en développement. Vite offre un serveur de développement ultra-rapide et une compilation optimisée.

Backend — Node.js + Express

Node.js permet d'utiliser JavaScript côté serveur, assurant une cohérence technologique avec le frontend. Express est un framework minimaliste et très répandu, idéal pour créer des API REST rapidement.

Base de données relationnelle — PostgreSQL

PostgreSQL a été retenu pour sa robustesse, sa conformité aux standards SQL et sa gestion avancée des types de données. Il stocke les données structurées : utilisateurs, menus, plats, commandes et avis.

Base de données non relationnelle — MongoDB

MongoDB permet de stocker des données flexibles et variables : l'historique des statuts de commande, les sessions utilisateurs et les statistiques du tableau de bord administrateur.

Authentification — JWT (JSON Web Token)

L'authentification par token JWT a été choisie pour sa nature stateless, évitant le stockage de sessions en base de données. Les tokens expirent après 24 heures pour des raisons de sécurité.

Déploiement

- Frontend : Vercel — déploiement automatique à chaque push sur la branche main
- Backend : Render — hébergement du serveur Node.js
- PostgreSQL : Render — base de données relationnelle en production
- MongoDB : MongoDB Atlas — base de données non relationnelle en production

2. Configuration de l'environnement de travail

Outils utilisés

Éditeur de code

Visual Studio Code (VS Code) version 1.133 — éditeur principal pour le développement frontend et backend.

Gestionnaire de versions

Git avec GitHub pour le versioning du code. Le dépôt public est accessible à l'adresse :

<https://github.com/gaetanlegranddev-svg/vite-et-gourmand>

Node.js

Version 24.18.0 — environnement d'exécution JavaScript côté serveur.

PostgreSQL

Version 18.1 — système de gestion de base de données relationnelle installé localement.

MongoDB

Version 8.3 — base de données non relationnelle installée localement.

Structure du projet

vite-et-gourmand/

├─ backend/

| └─ config/ → Configuration PostgreSQL

| └─ controllers/ → Logique métier des routes

| └─ middleware/ → Authentification JWT

| └─ models/ → Schémas MongoDB

| └─ routes/ → Définition des endpoints API

| └─ utils/ → Utilitaires (mailer, asyncHandler)

| └─ server.js → Point d'entrée du serveur

├─ src/

| └─ app/ → Composant principal App.tsx

| └─ components/ → Composants React (17 composants)

| └─ context/ → Contexte d'authentification

- | |— hooks/ → Hooks personnalisés (useFocusTrap)
- | |— services/ → Appels API (auth, menus, orders)
- | |— utils/ → Fonctions utilitaires partagées
- | |— styles/ → Feuilles de style

Workflow Git

Le projet suit le workflow **Git Flow** :

- **main** → branche principale, code stable et déployé
- **develop** → branche de développement, intégration des fonctionnalités
- **feature/xxx** → une branche par fonctionnalité, mergée vers develop après test

Variables d'environnement

Les informations sensibles sont stockées dans un fichier `.env` non commité sur GitHub :

PORT=3000

PG_HOST, PG_PORT, PG_DATABASE, PG_USER, PG_PASSWORD

MONGO_URI

JWT_SECRET

EMAIL_HOST, EMAIL_PORT, EMAIL_USER, EMAIL_PASS

3. *Modèle Conceptuel de Données*

Base de données relationnelle (PostgreSQL)

Table : users

Stocke tous les comptes utilisateurs (clients, employés, administrateurs).

- id (UUID, clé primaire)
- first_name, last_name (VARCHAR)
- email (VARCHAR, unique)
- phone, address (VARCHAR)
- password (VARCHAR, hashé avec bcrypt)
- role (ENUM : utilisateur, employee, admin)
- is_active (BOOLEAN)
- created_at (TIMESTAMP)

Table : menus

Représente les formules proposées par Vite & Gourmand.

- id (UUID, clé primaire)
- title, description (VARCHAR/TEXT)
- theme (ENUM : noel, paques, classique, evenement)
- regime (ENUM : classique, vegetarien, vegan, sans gluten, halal)
- price (DECIMAL), min_people (INT), stock (INT)
- active (BOOLEAN), min_order_days (INT)
- conditions, storage (TEXT)

Table : dishes

Représente les plats individuels (entrée, plat, dessert).

- id (UUID, clé primaire)
- name (VARCHAR)
- type (ENUM : entree, plat, dessert)

Table : allergens

Stocke les 14 allergènes réglementaires UE.

- id (UUID, clé primaire)
- name (VARCHAR, unique)

Table pivot : dish_allergens

Relation many-to-many entre plats et allergènes.

- dish_id (FK → dishes)
- allergen_id (FK → allergens)

Table pivot : menu_dishes

Relation many-to-many entre menus et plats.

- menu_id (FK → menus)
- dish_id (FK → dishes)

Table : orders

Enregistre chaque commande passée par un utilisateur.

- id (UUID, clé primaire)
- user_id (FK → users), menu_id (FK → menus)
- first_name, last_name, email, phone
- event_date (DATE), delivery_time (TIME)
- address, city (VARCHAR)
- in_bordeaux (BOOLEAN), distance_km (DECIMAL)
- people (INT)
- menu_subtotal, delivery_fee, discount, total (DECIMAL)
- notes (TEXT)
- current_status (ENUM : en attente, accepté, en préparation, en cours de livraison, livré, en attente du retour de matériel, terminée, annulée)
- has_equipment_loan (BOOLEAN)
- cancellation_reason, cancellation_contact_mode (VARCHAR)

Table : reviews

Stocke les avis clients après une commande terminée.

- id (UUID, clé primaire)
- order_id (FK → orders, unique)
- rating (INT, entre 1 et 5)
- comment (TEXT)
- validated (BOOLEAN)

- created_at (TIMESTAMP)

Relations principales

- Users → Orders : 1 à N (un utilisateur peut passer plusieurs commandes)
- Menus → Orders : 1 à N (un menu peut être commandé plusieurs fois)
- Menus ↔ Dishes : N à N via menu_dishes
- Dishes ↔ Allergens : N à N via dish_allergens
- Orders → Reviews : 1 à 0..1 (une commande peut avoir au maximum un avis)

Base de données non relationnelle (MongoDB)

Collection : status_history

Historise tous les changements de statut d'une commande.

- order_id (String), status (String), at (Date), note (String)

Collection : hour_slots

Stocke les horaires d'ouverture du lundi au dimanche.

- day (String), time (String)

Collection : sessions

Gère les tokens JWT avec expiration automatique (TTL index).

- user_id (String), token (String), expires_at (Date)

Collection : statistics

Stocke les statistiques de commandes par menu pour le dashboard admin.

- menu_id (String), menu_title (String), total_orders (Number), total_revenue (Number)

4. Diagrammes d'utilisation

Acteurs du système

- **Visiteur** — utilisateur non connecté
- **Utilisateur** — client connecté
- **Employé** — membre de l'équipe Vite & Gourmand
- **Administrateur** — Julie ou José, accès complet

Cas d'utilisation par acteur

Visiteur

- Consulter la page d'accueil
- Consulter tous les menus
- Filtrer les menus (prix, thème, régime, personnes)
- Voir le détail d'un menu
- Consulter les horaires d'ouverture
- Contacter l'entreprise via le formulaire
- Créer un compte
- Se connecter

Utilisateur (hérite du Visiteur)

- Passer une commande
- Consulter ses commandes
- Modifier une commande (statut "en attente" uniquement)
- Annuler une commande (statut "en attente" uniquement)
- Suivre le statut de sa commande
- Laisser un avis (commande "terminée" uniquement)
- Modifier son profil

Employé

- Consulter toutes les commandes
- Filtrer les commandes par statut ou client
- Avancer le statut d'une commande

- Annuler une commande (après contact client obligatoire)
- Gérer les menus et les plats
- Gérer les horaires d'ouverture
- Valider ou refuser les avis clients

Administrateur (hérite de l'Employé)

- Créer des comptes employés
 - Désactiver des comptes employés
 - Consulter les statistiques (commandes par menu, chiffre d'affaires)
 - Filtrer les statistiques par menu et période
-

5. Diagramme de séquence — Passage d'une commande

Utilisateur Frontend Backend API PostgreSQL Email

```
|      |      |      |      |
|--connexion-->|      |      |      |
|      |--POST /auth/login----->|      |
|      |<--token JWT-----|      |
|      |      |      |      |
|--commander-->|      |      |      |
|      |--POST /api/orders----->|      |
|      |      |--INSERT orders----->|
|      |      |<--order créée-----|
|      |      |--sendMail()----->|
|      |      |      | email envoyé
|      |<--201 Created-----|      |
|<--confirmation|      |      |      |
```

6. Documentation du déploiement

Architecture de production

Utilisateur



Vercel (Frontend React)

<https://vite-et-gourmand-red.vercel.app>



Render (Backend Node.js)

<https://vite-gourmand-backend.onrender.com>



Render MongoDB Atlas

PostgreSQL (Sessions, Stats)

Étapes de déploiement

Frontend (Vercel)

1. Connexion à Vercel avec GitHub
2. Import du repository vite-et-gourmand
3. Framework : Vite, Build Command : `npm run build`
4. Variable d'environnement : `VITE_API_URL`
5. Déploiement automatique à chaque push sur main

Backend (Render)

1. Création d'un Web Service sur Render
2. Root Directory : backend, Start Command : `node server.js`
3. Variables d'environnement configurées (PostgreSQL, MongoDB, JWT, Email)
4. Déploiement automatique à chaque push sur main

Base de données PostgreSQL (Render)

1. Création d'une instance PostgreSQL gratuite
2. Import du schéma via `psql -f database.sql`
3. Insertion des données de démo

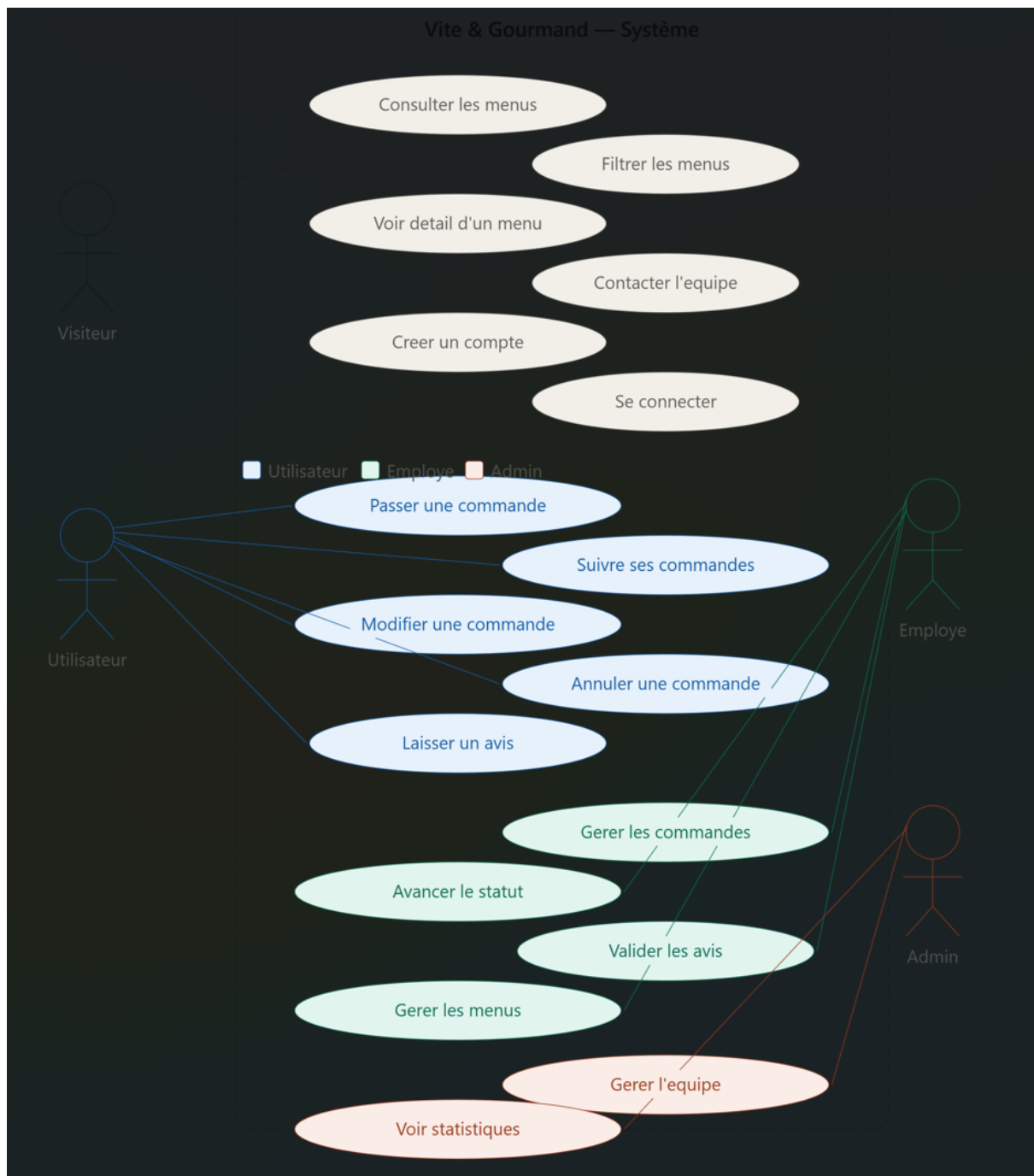
Base de données MongoDB (Atlas)

1. Création d'un cluster M0 gratuit
2. Configuration Network Access : 0.0.0.0/0
3. URI de connexion dans les variables d'environnement

Sécurité mise en place

- Mots de passe hashés avec **bcrypt** (10 rounds)
- Authentification par **JWT** avec expiration 24h
- **CORS** configuré pour autoriser uniquement le domaine Vercel
- Variables sensibles dans **variables d'environnement** (jamais dans le code)
- **SSL** activé pour la connexion PostgreSQL en production
- Validation des données côté backend avant insertion en BDD
- Middleware d'autorisation par rôle (utilisateur, employé, admin)
- Correction de la faille IDOR sur `/api/orders/:id/history` (vérification que l'utilisateur est propriétaire de la commande)
- Suppression des credentials en dur dans le code (mot de passe PostgreSQL, JWT Secret, mot de passe email)
- Messages d'erreur backend masqués pour éviter la divulgation d'informations sensibles
- Normalisation Unicode pour la validation des statuts de commande (gestion des accents)
- Stock des menus décrémenté automatiquement à chaque commande

Diagramme d'utilisation (Use Case)



7. Améliorations futures et limitations connues.

Limitations connues

Emails en production

Les emails de confirmation de commande et de bienvenue fonctionnent en environnement local. En production, Render bloque les connexions SMTP sortantes même sur le plan payant. La solution serait d'utiliser un service dédié comme Brevo ou SendGrid.

Premier chargement

Le backend Render s'endort après 15 minutes d'inactivité. Le premier chargement peut prendre 30 à 50 secondes.

Navigation par URL

L'application étant une SPA (Single Page Application), le bouton "précédent" du navigateur ne fonctionne pas comme attendu. L'intégration de React Router permettrait de gérer des URLs distinctes pour chaque page.

Encodage des caractères

Les insertions directes via psql sur Windows peuvent corrompre les caractères accentués. Solution: utiliser chcp 65001 avant chaque session psql.